

Implicit conversion traits and utility functions

- [Introduction](#)
- [Discussion](#)
- [Proposal Candidates](#)
- [Resolved Issues](#)
- [Proposed Resolution \[Exposition\]](#)
- [Feature-testing Macros](#)
- [Bibliography](#)
- [Partial Sample Implementation](#)

Introduction

This is an initial paper that focuses on some missing traits describing *implicit conversions* — related to the existing `is_convertible` trait — and on some utility function templates useful in that context of such conversions.

Albeit this proposal *could* have been a rather minimalistic proposal by suggesting only a single new trait `is_nothrow_convertible`, the author prefers to do it the harder way and to explore to some extent a set of reasonable utility additions around the same main topic, asking the committee for feedback to refine it in a second step.

By means of these suggested new facilities, this paper attempts to resolve the existing library issues [LWG 2040](#) and [LWG 2999](#) and to improve some other Standard Library specifications.

Discussion

The `is_convertible` trait is a quite interesting trait, because it describes **implicit conversions** (See also ¹ for a survey about implicit and explicit conversions). From a core language point of view, a different angle of looking at implicit conversions is to talk about **copy initialization**, because both meet each other in a variable definition of the form

```
T t = e;
```

Other initialization contexts that are described by **copy initialization** are aggregate member initialization, function return, and argument passing, to name just a few of them, which shows (a) the ubiquitous nature of implicit conversions and (b) the nearness to construction semantics, the latter being described in the most general (parenthesized) direct-initialization form by the `is_constructible` trait.

With these relations in mind, it seems rather unfortunate that the historic trait adjustment proposal [N3142](#) omitted a similar treatment of *triviality* and *nothrow* aspects of the `is_convertible` trait.

Therefore, this proposal suggests to add two new type traits `std::is_nothrow_convertible<From, To>` and `std::is_trivially_convertible<From, To>` to the header `<type_traits>`. In addition to that, this paper suggests to replace the existing pseudofunction `DECAY_COPY` by a real library template `std::decay_copy` added to header `<utility>` and a complete new conversion function `std::implicit_cast` added to header `<utility>` as well.

Albeit this paper provides a [Proposed Resolution](#) section, this one should be considered as expositional for the level of this initial paper revision to provide a rough picture of the intended amount of wording changes!

Proposal Candidates

1. `is_nothrow_convertible`

The `is_nothrow_convertible` trait

```
template <class From, class To>
struct is_nothrow_convertible;
```

is presumably one of the most often asked for traits related to `is_convertible`. One of the earliest official notes of the lack of that feature occurred during the publication of the [N3255](#) proposal that shortly before the C++11 finalization attempted to standardize a new Standard Library function template `decay_copy` as replacement for the existing `DECAY_COPY` pseudo-function. Among other reasons (especially the lateness of the feature request), this proposal failed, because it couldn't provide the correct conditional exception specification, concluding:

"What we would need is `std::is_nothrow_convertible`."

Shortly after C++11 standardization, [LWG 2040](#) was filed requesting the addition of `is_nothrow_convertible` and `is_trivially_convertible`, but is since then in a kind of zombie state.

There are several concrete use-cases for the `is_nothrow_convertible` trait:

1. It could be used to specify the correct `noexcept` expression for a `decay_copy` replacement template for `DECAY_COPY`, which would resolve one important aspect of [LWG 2999](#).
2. It could be used to restore the valueable (now conditionally) `noexcept` specification for several non-throwing `basic_string` functions that had been "string_view"-ified, as described by [LWG 2946](#). As example consider adjusting the currently suggested wording changes for the following `find` signature:

```
template <class T>
size_type find(basic_string_view<charT, traits> sv, const T& t, size_type pos = 0) const noexcept(see below);
```

-1- *Effects:* Creates a variable, `sv`, as if by `basic_string_view<charT, traits> sv = t;` and then determines the lowest position `xpos`, if possible, such that both of the following conditions hold: [...]

-2- *Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

-?- *Remarks:* This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false. The expression inside `noexcept` is equivalent to:

```
is_nothrow_convertible_v<const T&, basic_string_view<charT, traits>>
```

3. It could be used to define exception-correct — either by user-code or by the Standard Library — a fundamental `implicit_cast` utility function, see below.

2. is_trivially_convertible

The `is_trivially_convertible` trait answers the question whether during an implicit conversion as specified by `is_convertible`, only trivial functions are involved:

```
template <class From, class To>
struct is_trivially_convertible;
```

This trait requires compiler-support and it should therefore be sufficiently motivated. The `is_convertible<From, To>` semantics has very much similarities to `is_constructible<To, From>`, but it applies in cases where `is_constructible` doesn't work, for example the semantics of parameter passing or function return or aggregate member initialization. All of these situations can take advantage of reflecting upon trivial conversions.

For example, one could conditionally decide for a function parameter policy based on "by-value" or "by-reference" by considering whether the relevant initialization (either homogenous or heterogenous) involves only trivial operations or not.

The author asserts that once you have implemented a binary `is_trivially_constructible` intrinsic, you get the `is_trivially_convertible` intrinsic for a very cheap price, since the essential work is the same, except that you consider the initialization using implicit conversions instead of explicit ones. There are possible other candidates of "primitive" operations that would like to take advantage of a corresponding `is_trivially_xxx` traits, too, but it seems that implicit conversions are so widespread and in regard to their fundamental role comparable to `is_constructible` semantics that the lack of `is_trivially_convertible` feels a bit like a hole in the trait system.

3. decay_copy

The `decay_copy` function template can be written essentially as follows:

```
template <class T>
constexpr decay_t<T> decay_copy(T&& v) noexcept(is_nothrow_convertible_v<T, decay_t<T>>)
{
    return std::forward<T>(v);
}
```

It describes the implicit conversion of an object (or function) of type `T` to its decayed object form, that is, references and cv qualifiers are removed (in the "correct" order) and array/function decay is performed (if possible). This kind of functionality is basically everywhere needed, where you want to prevent dangling references by introducing an independent object of the given type with the same "value" as its source.

In particular, the Standard Library uses this kind of semantics when specifying the construction of a `std::thread` and `std::async` to copy the functor and its possibly bound arguments, because in both cases the actual usage of these objects happens in a different thread from the caller thread. In the absence of a "real" `decay_copy` function, the library currently uses an exposition-only pseudo-function `DECAY_COPY` with the semantics of a function template as shown above, except that it doesn't have an exception specification and does not support `constexpr`.

It is not required to specify a function template `decay_copy` including a conditional `noexcept` specifier, but it would be very advantageous for such a primitive operation.

One particular appealing characteristics of `decay_copy` is, that it is hard to use it wrong. This is so, because this function *never* returns references, so the risk of dangling references is reduced to the set of broken types `T` where even the construction of `decay_t<T>` would lead to an internal dangling reference.

4. implicit_cast

The `implicit_cast` function template can be written essentially as follows:

```
template <class T, class U>
constexpr T implicit_cast(U&& u) noexcept(is_nothrow_convertible_v<U, T>)
{
    return std::forward<U>(u);
}
```

It can be used wherever an implicit conversion should be part of a possibly multi-step operation. One can consider `implicit_cast` as a more general function than `decay_copy`, because we could replace a call of `decay_copy(t)` by `implicit_cast<decay_t<decltype(t)>>(t)`, but this relation would normally not be taking advantage of in real-world code, because calling `decay_copy(t)` directly is much more intuitive to write. Just to give a usage example, let's look at the following line of the specification of [LWG 2993](#):

-?- *Effects:* Creates a variable `r` as if by `T& r = std::forward<U>(u);`, then constructs a reference wrapper object that stores a reference to `r`.

which could be rewritten to

-?- *Effects:* Constructs a reference wrapper object that stores a reference to `implicit_cast<T&>(std::forward<U>(u))`.

In the particular case of `implicit_cast`, some bikeshedding smell is in the air, another naming candidates could be `implicit_convert`, for example. But according to the opinion of the author, the term `implicit_cast` is very popular and most public discussions of this function template has the here discussed semantics or very near to it. Among other examples, the Boost library provides a similar (albeit not identical) [implicit_cast template](#).

The differences of both definitions are that the Boost form of `implicit_cast` enforces that the actual conversion to the target type τ happens on the *function argument* side instead of within the function body suggested by the form shown above. This difference is relevant when considering the semantics of reference binding and of copy-elision.

1. If a conversion results in a reference target type that binds to a prvalue source, the danger of a dangling reference exists. In the Boost form, the temporary bound to the reference parameter of `implicit_cast` persists until the completion of the full-expression containing the call, while in the form shown above the temporary is destroyed at the end of the full-expression in the return statement.
2. If a conversion is performed where the target type is non-copyable and non-movable, the Boost form would make the code ill-formed, because there is no way for the compiler to perform copy-elision of the function argument that is returned by the function, but the above shown form will support non-copyable/non-movable types.
3. If the conversion throws an exception, a correspondingly `noexcept` specification based on `is_nothrow_convertible` would correctly describe the exception behaviour of `implicit_cast` only in the above presented form of `implicit_cast`, because for the Boost form the conversion happens during function parameter initialization, which is excluded from what an exception specification describes. So the Boost form actually consists of two steps: The function parameter initialization and the initialization of the return value by the function argument. Only the second step would normally be described by an exception specification.

Another way of looking at the problem of the possibility to produce dangling references is by comparing the behaviour of `implicit_cast` with that of a `std::function` instance that would return a reference in situations described in [LWG 2813 \("std::function should not return dangling references"\)](#) or in similar situations directly applied to `std::invoke` or *INVOKE*.

The author believes that generally the form of `implicit_cast` shown above is preferable over the Boost variant, assuming we can make its usage a bit safer for certain cases involving returning references. As the existing discussion of LWG [2813](#) shows, this is a much wider spread problem not specifically restricted to `implicit_cast`. There are basically two possible approaches to fix the dangling reference problem:

1. Attempt to define some conversion restrictions that would SFINAE out a library function template `implicit_cast`, similar to the work in progress for LWG [2813](#).
2. Define `implicit_cast` not as a library function template, but as a core language *type conversion operator* (such as `static_cast`). In such a case, any bound object to a reference would be still valid at the point, where the target variable has been initialized. In this case the operator presumably requires a different name, since public utility templates named `implicit_cast` are quite common.

The author would like to gather feedback from the committee regarding the preferred directions for `implicit_cast`.

Resolved Issues

If either of the proposed resolutions will be accepted, the following library issues will be resolved:

Number	Description
2040	Missing type traits related to <code>is_convertible</code>
2999	<code>§[thread.decaycopy]</code> issue

Proposed resolution [*Exposition*]

At some places below, additional markup of the form

[Drafting notes: whatever — end drafting notes]

is provided, which is *not* part of the normative wording, but is solely shown to provide additional information to the reader about the rationale of the concrete wording.

The proposed wording changes refer in all cases to [N4659](#).

1. Change 23.2.1 [utility.syn], header <utility> synopsis, as indicated:

```
[...]
// 23.2.5 [forward], forward/move
template <class T>
    constexpr T&& forward(remove_reference_t<T>& t) noexcept;
template <class T>
    constexpr T&& forward(remove_reference_t<T>&& t) noexcept;
template <class T>
    constexpr remove_reference_t<T>&& move(T&&) noexcept;
template <class T>
    constexpr conditional_t<
        !is_nothrow_move_constructible_v<T> && is_copy_constructible_v<T>, const T&, T&&>
        move_if_noexcept(T& x) noexcept;
template <class T>
    constexpr decay_t<T> decay_copy(T&& v) noexcept(is_nothrow_convertible_v<T, decay_t<T>>);
template <class T, class U>
    constexpr T implicit_cast(U&& u) noexcept(is_nothrow_convertible_v<U, T>);
```

2. At the end of 23.2.5 [forward] add the following series of paragraphs:

```
template <class T>
    constexpr decay_t<T> decay_copy(T&& v) noexcept(is_nothrow_convertible_v<T, decay_t<T>>);

-?- Effects: Equivalent to: return std::forward<T>(v);.

-?- Remarks: This function shall not participate in overload resolution unless is_convertible_v<T, decay_t<T>> is true. [Note: See 33.3.2.2 [thread.thread.constr] and 33.6.9 [futures.async] for examples of use. — end note]
```

[Drafting note: Below we use the more general term *reference-compatible* with instead of *reference-related* to, because `implicit_cast` can also be used to convert "noexcept function" to "function" — end drafting note]

```
template <class T, class U>
constexpr T implicit_cast(U&& u) noexcept(is_nothrow_convertible_v<U, T>);
```

-?- Effects: Equivalent to: `return std::forward<U>(u);`.

-?- Remarks: Let *v* be the type `remove_reference_t<U>`. This function shall not participate in overload resolution unless

- `is_convertible_v<U, T>` is true, and
- If *T* is type "reference to *S*", *v* is a class type or *S* is reference-compatible (11.6.3 [dcl.init.ref]) with *v*.

3. Change 23.15.2 [meta.type.synop], header <type_traits> synopsis, as indicated:

```
namespace std {
[...]
// 23.15.6 [meta.rel], type relations
template <class T, class U> struct is_same;
template <class Base, class Derived> struct is_base_of;
template <class From, class To> struct is_convertible;
template <class From, class To> struct is_trivially_convertible;
template <class From, class To> struct is_nothrow_convertible;
[...]
```

```
// 23.15.6 [meta.rel], type relations
template <class T, class U> inline constexpr bool is_same_v
= is_same<T, U>::value;
template <class Base, class Derived> inline constexpr bool is_base_of_v
= is_base_of<Base, Derived>::value;
template <class From, class To> inline constexpr bool is_convertible_v
= is_convertible<From, To>::value;
template <class From, class To> inline constexpr bool is_trivially_convertible_v
= is_trivially_convertible<From, To>::value;
template <class From, class To> inline constexpr bool is_nothrow_convertible_v
= is_nothrow_convertible<From, To>::value;
[...]
```

4. Change 23.15.6 [meta.rel], Table 44 — "Type relationship predicates", as indicated:

Table 44 — Type relationship predicates

Template	Condition	Comments
...		
template <class From, class To> struct is_convertible;	see below	From and To shall be complete types, arrays of unknown bound, or cv void types.
template <class From, class To> struct is_trivially_convertible;	<code>is_convertible_v<From, To></code> is true and the conversion, as defined by <code>is_convertible</code> , is known to call no operation that is not trivial. ([basic.types]..[special]).	From and To shall be complete types, arrays of unknown bound, or cv void types.
template <class From, class To> struct is_nothrow_convertible;	<code>is_convertible_v<From, To></code> is true and the conversion, as defined by <code>is_convertible</code> , is known not to throw any exceptions. ([expr.unary.noexcept]).	From and To shall be complete types, arrays of unknown bound, or cv void types.
...		

5. Remove the complete sub-clause 33.2.6 [thread.decaycopy] as indicated:

~~33.2.6 decay_copy [thread.decaycopy]~~

~~In several places in this Clause the operation `DECAY_COPY(x)` is used. All such uses mean call the function `decay_copy(x)` and use the result, where `decay_copy` is defined as follows:~~

```
template <class T> decay_t<T> decay_copy(T&& v)
{ return std::forward<T>(v); }
```

6. Change 33.3.2.2 [thread.thread.constr] as indicated:

```
template <class F, class... Args> explicit thread(F&& f, Args&&... args);
```

-3- Requires: *F* and each *T_i* in *Args* shall satisfy the MoveConstructible requirements. `INVOKE(DECAY_COPY(decay_copy(std::forward<F>(f)), DECAY_COPY(decay_copy(std::forward<Args>(args))...))` (23.14.3 [func.require]) shall be a valid expression.

-4- Remarks: This constructor shall not participate in overload resolution if `decay_t<F>` is the same type as `std::thread`.

-5- *Effects:* Constructs an object of type `thread`. The new thread of execution executes `INVOKE(decay_copy(decay_copy(std::forward<F>(f)), decay_copy(decay_copy(std::forward<Args>(args))...))` with the calls to `decay_copy` being evaluated in the constructing thread. Any return value from this invocation is ignored. [Note: This implies that any exceptions not thrown from the invocation of the copy of `f` will be thrown in the constructing thread, not the new thread. — end note] If the invocation of `INVOKE(decay_copy(decay_copy(std::forward<F>(f)), decay_copy(decay_copy(std::forward<Args>(args))...))` terminates with an uncaught exception, `terminate` shall be called.

[...]

7. Change 33.6.9 [futures.async] as indicated:

```
template <class F, class... Args>
future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
    async(F&& f, Args&&... args);
template <class F, class... Args>
future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
    async(launch_policy, F&& f, Args&&... args);
```

-2- *Requires:* `F` and each `Ti` in `Args` shall satisfy the `MoveConstructible` requirements, and

```
INVOKE(decay_copy(decay_copy(std::forward<F>(f)),
    decay_copy(decay_copy(std::forward<Args>(args))...)) // see 23.14.3 [func.require], 33.3.2.2 [thread.thread.constr]
```

-3- *Effects:* [...] The further behavior of the second function depends on the policy argument as follows (if more than one of these conditions applies, the implementation may choose any of the corresponding policies):

(3.1) — If `launch::async` is set in policy, calls `INVOKE(decay_copy(decay_copy(std::forward<F>(f)), decay_copy(decay_copy(std::forward<Args>(args))...))` (23.14.3 [func.require], 33.3.2.2 [thread.thread.constr]) as if in a new thread of execution represented by a thread object with the calls to `decay_copy` being evaluated in the thread that called `async`. Any return value is stored as the result in the shared state. Any exception propagated from the execution of `INVOKE(decay_copy(decay_copy(std::forward<F>(f)), decay_copy(decay_copy(std::forward<Args>(args))...))` is stored as the exceptional result in the shared state. The thread object is stored in the shared state and affects the behavior of any asynchronous return objects that reference that state.

(3.2) — If `launch::deferred` is set in policy, stores `decay_copy(decay_copy(std::forward<F>(f))` and `decay_copy(decay_copy(std::forward<Args>(args))...` in the shared state. These copies of `f` and `args` constitute a deferred function. Invocation of the deferred function evaluates `INVOKE(std::move(g), std::move(xyz))` where `g` is the stored value of `decay_copy(decay_copy(std::forward<F>(f))` and `xyz` is the stored copy of `decay_copy(decay_copy(std::forward<Args>(args))...` [...]

(3.3) — [...]

Feature-testing Macros

For the purposes of SG10, this paper recommends the macro name `__cpp_lib_additional_convertible_traits`.

Bibliography

[N3255 Lawrence Crowl, Daniel Krügler: "C++ Decay Copy"](#)

[P0705R0 Tony Van Eerd: "Implicit and Explicit Conversions"](#)

Partial Sample Implementation

Example implementation for the `is_nothrow_convertible` type trait and the constrained `decay_copy` function template.

```
#include <type_traits> // std::enable_if, ...
#include <utility>      // std::forward, std::declval

namespace xstd {

namespace details {

template <class From, class To, bool =
    std::disjunction<
        std::is_void<From>, std::is_function<To>, std::is_array<To>
    >::value
>
struct do_is_nothrow_convertible
{
    using type = std::is_void<To>;
};

struct do_is_nothrow_convertible_impl
{
    template <class To>
    static void test_aux(To) noexcept;

    template <class From, class To>
    static std::bool_constant<noexcept(test_aux<To>(std::declval<From>()))>
    test(int);

    template <class, class>
    static std::false_type
    test(...);
};
```

```

template <class From, class To>
struct do_is_nothrow_convertible<From, To, false>
{
    using type = decltype(do_is_nothrow_convertible_impl::test<From, To>({}));
};

} // details

template <class From, class To>
struct is_nothrow_convertible :
    details::do_is_nothrow_convertible<From, To>::type
{ };

template <class From, class To>
inline constexpr bool is_nothrow_convertible_v
    = is_nothrow_convertible<From, To>::value;

template <class T>
constexpr
std::enable_if_t<
    std::is_convertible_v<T, std::decay_t<T>>,
    std::decay_t<T>
>
decay_copy(T&& v) noexcept(is_nothrow_convertible_v<T, std::decay_t<T>>)
{
    return std::forward<T>(v);
}

} // xstd

```